

SISTEMAS DISTRIBUÍDOS

Aula 19 — Algoritmos Distribuídos II: Snapshot Distribuído e Detecção de Terminação

6º Semestre CC/ES-SI · Prof. Leandro Borges · 20/10

Agenda da Aula

- 1 O problema do snapshot distribuído
- 2 Por que um snapshot ingênuo é inconsistente
- 3 Chandy-Lamport: marcadores e captura de canais
- 4 O que um snapshot consistente garante
- 5 Detecção de terminação distribuída: o problema
- 6 Dijkstra-Scholten e wave algorithms (anel de tokens)

Snapshot Distribuído: O Problema

O que é o estado global de um sistema distribuído

É a combinação do estado interno de cada processo com o conteúdo de cada canal de comunicação — as mensagens já enviadas, mas ainda não recebidas. Conhecer esse estado global é essencial para depuração, checkpointing, detecção de deadlock e coleta de lixo distribuída.

Por que não basta simplesmente “tirar uma foto”

Não existe relógio global compartilhado nem forma de pausar todos os processos ao mesmo tempo. Se cada processo registra seu estado em um instante diferente do tempo real, mensagens podem ser contadas duas vezes, somem na captura, ou geram uma sequência de eventos que nunca poderia ter ocorrido de fato.

Por Que um Snapshot Ingênuo Falha

Mensagem “fantasma”

Se o receptor registra seu estado antes do remetente, e a mensagem já foi enviada mas ainda não chegou, o snapshot mostra a mensagem chegando “do nada” — recebida sem nunca ter sido enviada, o que é logicamente impossível.

Mensagem “perdida”

Se o remetente registra seu estado depois de enviar, mas o receptor já registrou o dele antes de receber, o snapshot mostra a mensagem enviada mas nunca recebida — ela desaparece da “foto” mesmo estando em trânsito na rede.

Chandy-Lamport: Marcadores e Captura de Canais

Iniciando o snapshot

Qualquer processo pode iniciar: registra seu próprio estado local e envia um marcador (marker) por todos os seus canais de saída, antes de qualquer outra mensagem que seja gerada depois da captura começar.

Recebendo um marcador

Ao receber o primeiro marcador em qualquer canal, o processo registra seu estado local (se ainda não fez) e passa a gravar as mensagens que chegam por esse canal até receber um marcador nele — isso define o estado do canal na foto.

O Snapshot Global Resultante: O Que Ele Garante

Propriedade	O que significa
Corte consistente	Se um evento “depois” está no snapshot, todo evento “antes” dele (na ordem causal) também está — nenhum efeito aparece sem sua causa.
Não é um instante real	Combina estados registrados em momentos diferentes do tempo físico; representa um estado global que poderia ter ocorrido, não necessariamente um que ocorreu de fato.
Sem parar o sistema	Processos continuam executando normalmente durante a captura — o algoritmo roda concorrentemente com a computação, sem coordenação centralizada.
Canais capturados	As mensagens em trânsito no momento da captura ficam registradas no estado do canal — nada se perde, nada duplica.

Detecção de Terminação Distribuída

O problema

Um cálculo distribuído termina quando todos os processos estão passivos (sem trabalho a fazer) e não há nenhuma mensagem em trânsito entre eles. O desafio é que nenhum processo isolado sabe disso: um processo pode estar passivo agora e ser reativado por uma mensagem que ainda está a caminho vinda de outro processo.

Por que é difícil sem relógio global ou coordenador

Sem um observador central que veja todos os processos e canais ao mesmo tempo, e sem um relógio compartilhado que diga “agora”, afirmar que “o cálculo terminou” exige um protocolo adicional — os próprios processos precisam colaborar para detectar coletivamente uma condição que nenhum deles observa sozinho.

Dijkstra-Scholten vs. Wave Algorithms (Anel de Tokens)

Aspecto	Dijkstra-Scholten	Wave / Anel de Tokens
Estrutura	Árvore dinâmica enraizada no processo iniciador, construída pelas próprias mensagens de computação	Um token circula por um anel lógico (ou percorre todos os processos) coletando informação de estado
Como detecta	Cada processo conta mensagens enviadas e reconhecidas (ack); a raiz sabe que terminou quando a árvore “colapsa” até ela	O token só reporta término quando dá uma volta completa encontrando todos os processos passivos e nenhuma mensagem pendente
Overhead	Um reconhecimento (ack) extra por mensagem de aplicação enviada	Mensagens de token adicionais, à parte da computação
Ideia central	Rastrear localmente a relação de dependência entre quem enviou trabalho para quem	Fazer uma “onda” de verificação percorrer repetidamente todo o sistema até confirmar quietude global

Onde Isso é Usado na Prática

Checkpointing e recuperação de falhas

Sistemas de computação de longa duração (simulações científicas, processamento em cluster) usam snapshots consistentes como o de Chandy-Lamport para salvar o estado do sistema periodicamente e retomar de onde parou após uma falha, sem reprocessar tudo do zero.

Coleta de lixo distribuída e frameworks de processamento em massa

Detecção de terminação é o que permite a um framework como o MapReduce (ou a um coletor de lixo distribuído) saber com segurança quando uma fase de processamento acabou e é seguro avançar para a próxima, mesmo com milhares de processos trabalhando de forma independente.

Próxima Aula

Algoritmos Distribuídos III: Replicação e Consistência de Dados

22/10 — como manter réplicas de dados consistentes entre si.

Referências desta Aula

- CHANDY, K. M.; LAMPORT, L. Distributed Snapshots: Determining Global States of Distributed Systems. ACM Transactions on Computer Systems, v. 3, n. 1, 1985.
- DIJKSTRA, E. W.; SCHOLTEN, C. S. Termination Detection for Diffusing Computations. Information Processing Letters, v. 11, n. 1, 1980.
- TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.